UNIVERIST OF RWANDA
CST-RMA
COMPUTER ENGINEERING
LEVEL 3

ON MARCH 29, 2026

REG NO: 223027475

FLUTTER LAB 4: CONSUMING APIs IN FLUTTER

Introduction

This lab focuses on building a Flutter mobile application that communicates with an external API. The goal is to understand how Flutter interacts with a backend system by sending and receiving data over the internet.

In this project, a test API from JSONPlaceholder is used to simulate real server behavior. The application developed is called **"Posts Manager"**, which allows users to perform CRUD operations (Create, Read, Update, Delete) on posts.

Application Description

The mobile application enables staff members to:

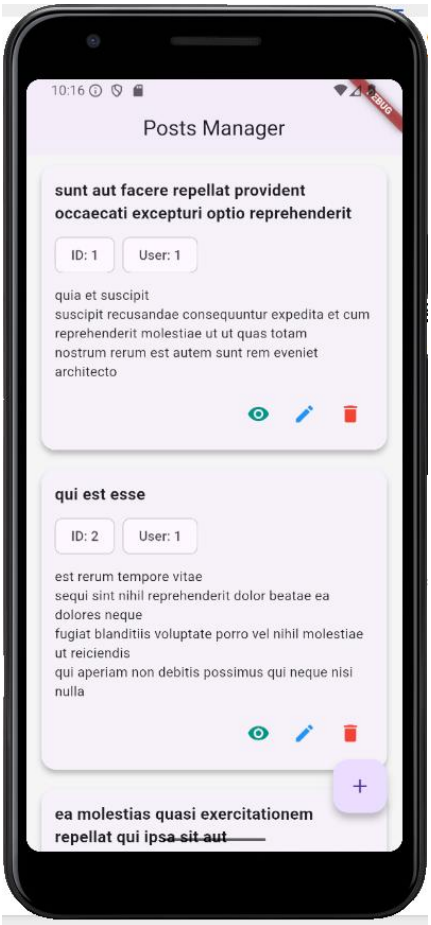
- View all available posts
- Read the details of a post
- Create a new post
- Edit an existing post
- Delete a post

The app communicates with the following API:
https://jsonplaceholder.typicode.com/posts

Application Screens (UI Design)

1. Home Screen (Posts List)

- Displays all posts fetched from the API
- Uses a `ListView` to show titles and short descriptions
- Includes a **Floating Action Button (+)** to add a new post
- Each post has options to:
    - View details
    - Edit
    - Delete



2. View Post Screen (Details Screen)

- Displays full details of a selected post
- Shows:
    - Post ID
    - Title
    - Body content
- Provides a clear and readable layout for users

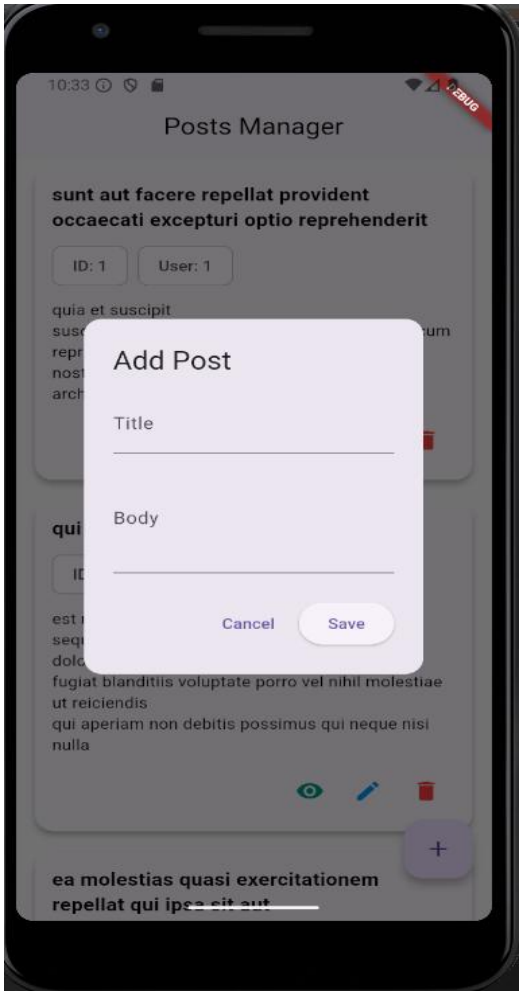
---

3. Create Post Screen

- Allows users to add a new post
- Contains:

    - Input field for title
    - Input field for body
    - Submit button

- Sends a **POST request** to the API



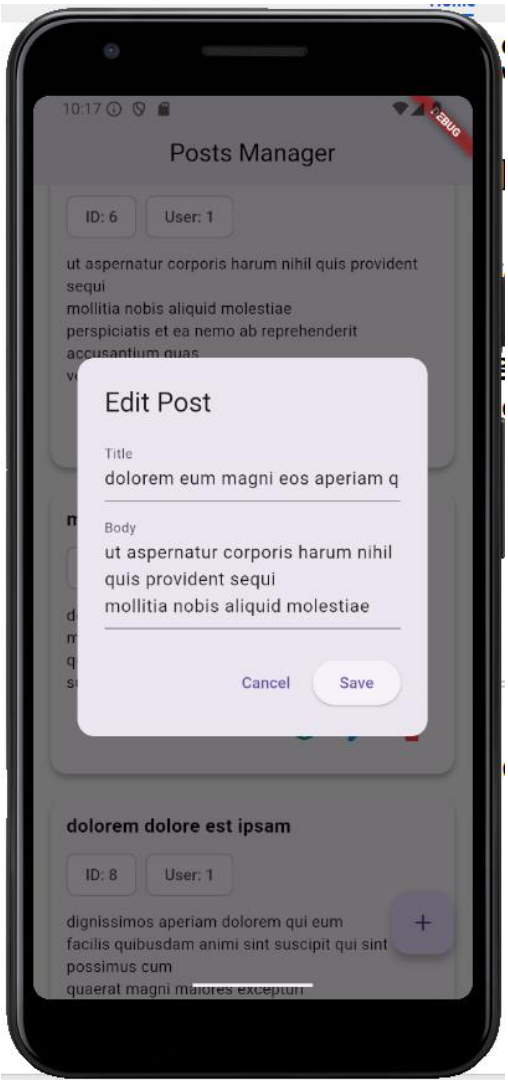
---

4. Edit Post Screen

- Allows users to update an existing post
- Pre-fills fields with existing data
- Users can modify:

    - Title
    - Body

- Sends a **PUT request** to update the post



5. Delete Post Feature

- Users can delete a post using a delete button
- Sends a **DELETE request** to the API
- The UI updates immediately after deletion

1. Dependencies Used and Why

The http dependency was used in this project:

http package

22302 7475

LAB 4

① In this project I used the following dependency:

→ http : it is the package used to enable Communication between the flutter application and the JSON place-holder API. it allows sending and receiving data through HTTP requests such as GET (to fetch posts), POST (to create posts), PUT (to update posts) and DELETE (to remove posts).

→ Other built-in dependencies are:
- flutter (SDK)
- dart: convert.

---

② API exceptions can occur when the application Communicates with a server. These errors Can be handled using the following methods.

① using try-catch Block.

- try-catch block is used to handle network-related errors such as no internet connect or request failure:

Why it is important

The http package enables communication between the Flutter application and the remote server, making it possible to retrieve and send data.

2. Handling API Exceptions

Handling API errors is very important to prevent the app from crashing and to improve user experience.

Common types of errors and how they are handled:

1. Network Errors

- Occur when there is no internet connection
- Handled using try-catch blocks

2. Invalid Responses

- Occur when the server returns an unexpected response
- Handled by checking `response.statusCode`

3. Timeout Errors

- Occur when the server takes too long to respond
- Can be handled using timeout functions

4. Invalid Data

- Occur when JSON data is not in expected format
- Handled using proper parsing and validation

```
try {
    // API request
  } catch (e) {
    print (e);
  }
```

(ii). Checking HTTP status codes

after making a request, the responce status code is checked to determine if the request was successful.

```
if (responce. statusCode == 200) {
    // success
  } else {
    throw Exception ("Failed to load data");
  }
```

(iii) Handling invalid data.

The application ensures that the data received from the API is valid before using it. JSON data should be properly decoded to avoid errors.

(IV). Displaying Errors in the UI

Errors can be shown to the user using widgets like FutureBuilder.

```
if (snapshot.hasError) {
    return Text("Error: ${snapshot.error}");
}
```

(V). Handling Loading State

```
if (snapshot.connectionState == ConnectionState.waiting)
{
    return CircularProgressIndicator();
}
```

(3) • What is a future?

Future is an object that represents a value that will be available later.
In flutter, Api calls time to complete, so they return a future instead of immediate data.

• What is future builder?

The Future builder widget listens to a future and rebuilds the UI depending on its state.
it automatically handles three states:

3. Future Widget Explanation

In Flutter, API calls are asynchronous, meaning they take time to complete.

To handle this, the **FutureBuilder** widget is used.

What is FutureBuilder?

FutureBuilder is a widget that builds the UI based on the result of a Future (asynchronous operation).

How it works:

It has three main states:

1. Loading State

- When data is still being fetched
- Shows a loading indicator (`CircularProgressIndicator`)

2. Error State

- If something goes wrong
- Displays an error message

3. Data State

- When data is successfully fetched
- Displays the posts in a list

Important Note

The UI (ListView) depends on the Future because it only displays posts after the API call is completed.

I. waiting → while the future is still running.
II. Error → if the future fails.
III. Completed → when the Future successfully returns data.

- How I used it:

I've used FutureBuilder to fetch posts from the Api and display them in a Listview.

```
FutureBuilder (
    future: fetchPosts(),
    builder: (Context, Snapshot) {
        if (Snapshot.connectionState == ConnectionState.waiting) {
    return Center (child: CircularProgressIndicator ());
    } else if (Snapshot.hasError {" ));
    } else {
    final posts = Snapshot.data!;
    return Listview.builder (
      itemCount: posts.lenght,
      itemBuilder: (Context, index) {
          return ListTile (
            title: Text (posts[index].title),
            Subtitle: Text (posts[index].body),
```

```
    ), },
    ); } }, )
```

```
);
},
);
}
},
)
```

- Parts of the UI that depends on the future
  1. Loading Indicator - shows while waiting for the API response.
  2. Error message - shows if there is a network or data Error.
  3. posts List - Shows the fetched posts once the future Completes successfully.

- The Future handles the asynchronous API request, and Future Builder ensures the UI updates automatically based on the whether the data is Loading, has an error or is successfully retrieved.

Conclusion

This lab helped in understanding how Flutter applications interact with APIs. It demonstrated how to perform CRUD operations using HTTP requests and how to handle asynchronous data using FutureBuilder.

It also emphasized the importance of handling errors properly to ensure a smooth user experience.